Reinforcement Learning

framework -> Markov-decision-process

- RL is a type of ML where an Agent learns through trial and error by Interacting with an environment.



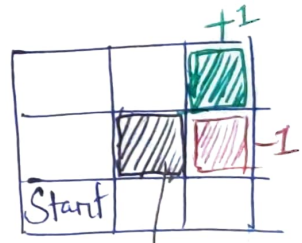
many Iteration machine becomes learned strongly.

- Agent receives rewards (+ve or -ve) for its actions, Aiming to maximize its long-term reward.
- RL is well suited for Problems with complex or dynamic environment where providing explicit instruction is critical / Impractical.

Markov Decision Processes (MDPs)

↳ it is a mathematical framework with certain constraint for modelling decision-making in sequential, stochastic (Probabilistic / random) environments.

Characteristic of MDP



- **States (S):** Set of all the Possible States the environment can be.

Possible state machine can achieve = Total state. (3x3 - 1 Obstacle) = 8 States

- **Action (A):** Set of Possible action can take by the Agent (Left, Right, Up, Down)
- **Transition Probability (P):** function $T(S, a, S')$ → Probability of transitioning from one State to another given an action.

State, action, next state

$P(s'/s,a)$ Probability of going into another State given current State & action is already happen.

- **Reward Function (R)**: immediate reward received by the agent for taking an action in a State.
  - $R(s,a)$ (take action over a State) → get Reward
- **Discount factor (γ)**: future Rewards relative to immediate rewards.
- **policy (π)**: It is the Solution to all this Constraints of the Markov Decision Process Model.
  - (machine Learns / make a policy) → (π) Solution
  - (S, A, R, P, γ) Problems

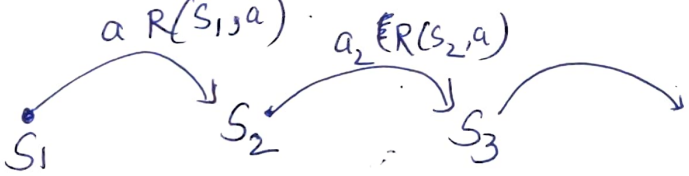
→ Fundamental Equation of RL

- **Bellman Equation**: fundamental equation in dynamic programming that relates the value of a State to the expected value of future rewards obtained from taking an Optimal action.

Value function $V(S)$ (value): $\max_{a}\left(R(S,a) + \gamma \sum_{s'} T(s,a,s')\,V(s')\right)$

(γ: Discount factor)





Q-Value Function ($Q(s,a)$) (Quality function)

$$V(S) = \max_{a_1}\left(R(S_1,a_1) + \gamma \sum_{S_2} T(S_1,a_1,S_2) \max_{a_2} R(S_2,a_2) + \gamma \sum_{S_3} T(S_2,a_2,S_3) \dots\right)$$

$$Q(s,a) = R(S_1,a) + \gamma \sum T(S_1,a_1,S_2) \max_{a'} Q(s',a')$$

- Value function ($V(s)$): Expected discounted future rewards starting from state (S), following a specific policy (π).
- Q-Value function ($Q(s,a)$): Expected discounted future rewards starting from state S, taking action a, and following Specific policy (π) thereafter.

Policy (π) : Mapping from states to actions, defining the agent's decision-making strategy.

Policy Evaluation : Evaluating the Value of a given Policy.

Policy Improvement : Finding better Policy than the Current One. Aiming for higher expected rewards.

Model based and Model Free Methods :

RL → machine Learns itself, A reward function associated with every action / stages.

Model based :-



Agent has the Complete Map/detailed map of Environment, So it can Plan its action based on the map.

Model Free :- Agent Learns directly through Exploration & iteraction with the Environment without having an explicit Knowledge.

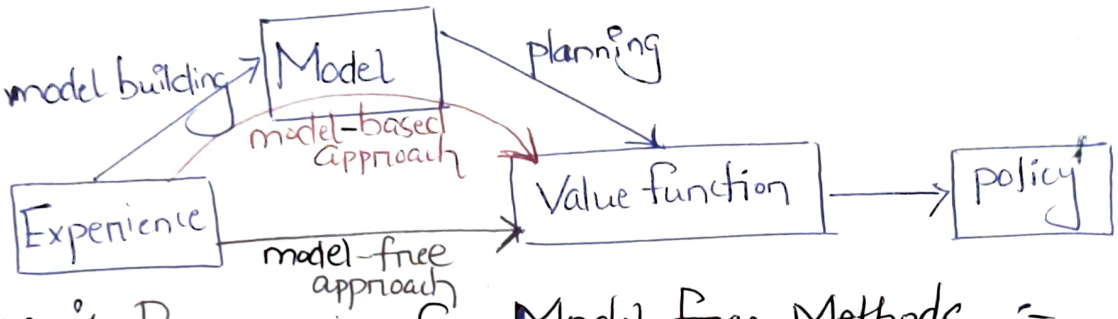
In Model based Methods :

- Learning a model of the environment. (transition Probability & Reward) (model has complete understanding of the environment previously.)
- Use the Model to plan & make decision.
- Computationally Expensive as in complex Environments machine has to first understand the whole Environments

Model-free Methods :

we do not has preconfiguration ready with us.

- machine Learns directly by interaction with the Environment.
- Do not require explicit modeling of the Environment.
- So it is more flexible and adaptable to changing environments.



Dynamic Programming for Model-free Methods :-

→ Dynamic Programming approaches to find the Optimal (π) policy.

it generate Value Function & Policy (π) at each stage Iteratively. Iterate previously as it has no Knowledge of Environment

also need Knowledge of Environment dynamics (because it depends on Transition Probability & Rewards).

① 'Q-Learning :

② SARSA : (State action Reward State action)

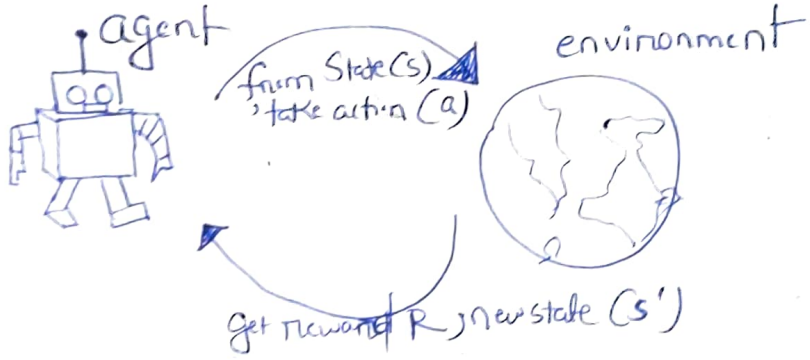
→ Estimate Q Value function : (which represents the expected Long term reward starting from state and taking a specific action.

it iteratively Updates the Qvalue estimates by using bellman equation and exploration-exploitation trade-off has to done.

Trade-off balances the need to explore new actions to find potentially better options with the desire to exploit what has already being learned & maximized (immediate rewards such)

② SARSA : it also Learns Q-value function.

Instead of updating Q-value based on Maximum future Reward. it Updates Q-value for the State-Action Pair visited after taking an action and receiving a Reward. This allows SARSE to Learn from Immediate experiences & Potentially converges faster than Q-Learning in Certain Situations.

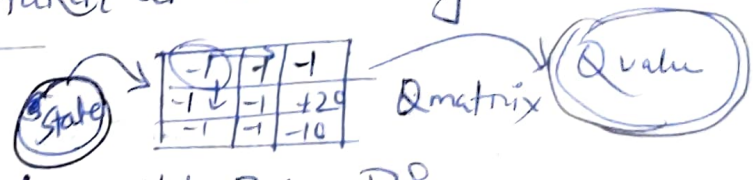


Deep Reinforcement Learning :- (Deep Q-Learning)

Q Learning : Q table → based on Q-table machine Learns What action to be taken at what Stage.

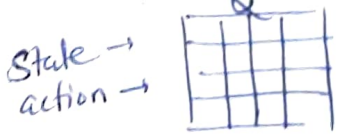
	A_1	A_2	A_3
S_1	Q	Q	Q
S_2	Q	Q	Q
S_3	Q	Q	Q

Q value for State Action Pair.



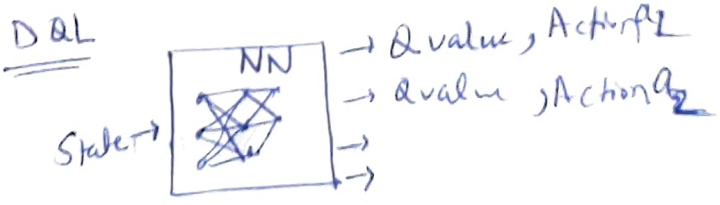
Deep Q Learning :

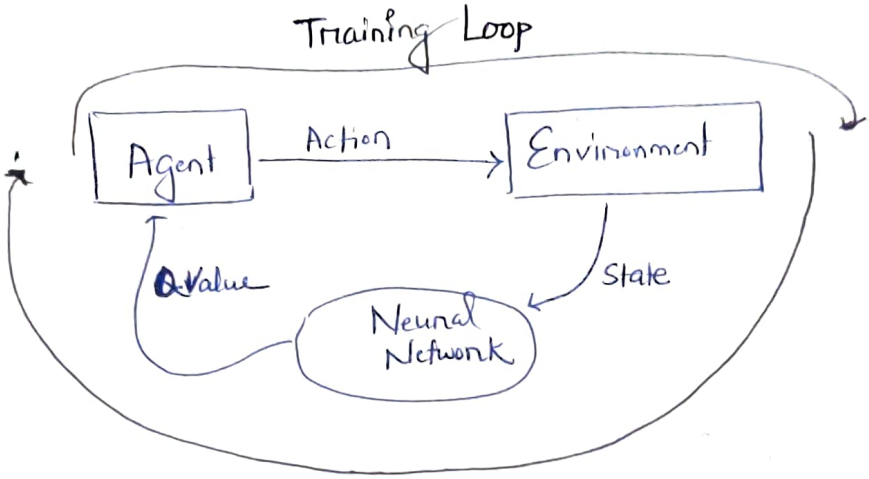
for Large state space.



- Deep Q-Learning extends Q-Learning using neural networks to approximates Q-values.
- it is suitable for environments with Large or Continious State spaces.
- Q-values are approximated instead of stored in a table.
- train a function so that machine takes action based on trained parameters.

DQL





Architecture of DQ-learning (DQL)

Step(1): Data Collection →

Step(2): Train The Model

S_1	S_2	S_3
	-1	-1
-1	-1	-1
-1	-10	+10

Experience Replay Buffer

$(S_1, \text{right}, -1, S_2)$

$(S_2, \text{right}, +1, S_3)$

(data - collection)

Q Network

Data Collected →

(O/p neurons = # of actions available for the Agent)

4 neurons ← (Left, Right, Up, Down)

Current Conciousness of The Agent.

$Q(S, \text{right}) = 0.67$

$Q(S_2, \text{Left}) = 1.36$

$Q(S_1, \text{Up}) = -1.63$

$Q(S, \text{Down}) = 1.2$

Target Network

Collecting Data →

-1.6

1.2

1.3

1.4

(1.6 - 1)

Reward

Compare

Loss

Update the Q Network (backPropagation)

$$Q(s,a) \leftarrow (1-\alpha)\cdot Q(s,a) + \alpha\cdot\left(\pi + \gamma \cdot \max_{a'} Q(s',a')\right) \quad Q(s,a) \leftarrow (1-\alpha)\cdot Q(s,a) + \alpha\cdot(\pi + \gamma \cdot \max_{a'} Q(s',a'))$$

$Q(s,a)$ - Q value for taking an action a in state s.

π - immediate reward's

s' - Next State after taking an action a in state s

α : Learning rate, determine how much to Update Q value

γ - Discount factor, determines importance of future rewards.

→ Reinforcement_learning Projects with OpenAI - gym

`Py -m venv .mynewenv`

`.\mynewenv\Scripts\Activate.ps1`

base → (.mynewenv)

Python 3.10.11

↳ `pip install gym==0.25.2 tensorflow==2.12.0 Keras-rl2 numpy pygame`

↳ `pip list`

Cmd

```
D:
cd D:\Reinforcement_Learning
code .
```

Ctrl + Shift + ` — New Terminal